MERIDIAN CAMPUS PORTAL

Web Application Security Assessment
Vulnerability Findings & Secure-Build Remediation Report

BUSINESS CONFIDENTIAL

Target (vulnerable build): http://meridian.tn:3000

Target (hardened build): http://meridian.tn:3001

Application: Meridian University — Student Portal (Flask / SQLite / Ed25519 / Ethereum)

Prepared by: Ahmed Rafraf

Date: 11 October 2026 · **Version:** 1.0

Confidentiality Statement

This document contains proprietary and confidential information relating to a security assessment performed against the **Meridian University Student Portal**, a purpose-built laboratory application developed for educational study of common web-application security flaws and their cryptographic and blockchain-based remediations. It is intended solely for the author and the module supervisors. Duplication or redistribution requires the author's consent.

Disclaimer

A penetration test is a snapshot in time. The findings and recommendations reflect the state of the application during the assessment and not any changes made afterwards. The Meridian Campus Portal is an intentionally insecure training application; all testing was conducted on a local instance owned and operated by the tester, with no impact on third parties. The application ships as two builds from a single code base — a **vulnerable** build (every flaw tagged `# VULN:`) and a **secure** build (each matching fix tagged `# SECURE:`) — so each weakness can be exploited, then re-tested against the hardened version to confirm the control works.

Contact Information

Name	Role	Contact
Ahmed Rafraf	Student / Tester	mohamedrafraf99@gmail.com
Module Supervisor	Reviewer	TEK-UP — Web Security Module

Table of Contents

1. Assessment Overview

2. Test Environment Setup — Local DNS Mapping for Burp Suite Interception

3. Finding Severity Ratings & Risk Factors

4. Scope

5. Executive Summary

6. Vulnerability Summary & Report Card

7. Security Architecture of the Hardened Build (Cryptography & Blockchain)

8. Technical Findings

- WAPT-001 — SQL Injection: Authentication Bypass (Critical)
- WAPT-002 — SQL Injection: UNION-based Hash Exfiltration (Critical)
- WAPT-003 — Broken JWT: Admin Token Forgery (Critical)
- WAPT-004 — IDOR: Private Chats & Student Documents (High)
- WAPT-005 — Mass Assignment: Privilege Escalation at Registration (High)
- WAPT-006 — Cryptographic Failure: Unsalted MD5 Password Storage (High)
- WAPT-007 — Cryptographic Failure: Unsigned / Un-anchored Certificates (High)
- WAPT-008 — Data Integrity Failure: Silent Grade Tampering (High)
- WAPT-009 — Mass Assignment: Privilege Escalation from Profile Update (High)
- WAPT-010 — Unrestricted File Upload → Stored XSS (High)
- WAPT-011 — Path Traversal: Local File Disclosure (Critical)

Appendix A — Methodology & Tools

Appendix B — Cryptographic & Blockchain Reference

1. Assessment Overview

This assessment evaluated the security posture of the Meridian University Student Portal, a role-based campus application (students, professors, administrators) covering authenticated login, grade and certification management, private messaging, administrative document requests and an administrative user-management console. Testing followed an industry-standard methodology aligned with the **OWASP Web Security Testing Guide (WSTG)** and the **OWASP Top 10 (2021)**, supplemented by manual exploitation.

The engagement had a second objective beyond discovery: to verify that the project's **secure build** genuinely neutralises each flaw. Particular attention was paid to the two defence mechanisms that distinguish this application from a conventional web app — a **digital-signature and encryption layer** (Ed25519, AES-256-GCM, PBKDF2) and a **blockchain integrity ledger** (a Solidity smart contract on a local Ethereum node that anchors grades and certificates). The testing activities followed four phases:

- **Planning** — define objectives, scope and rules of engagement.
- **Discovery** — reconnaissance, fingerprinting and enumeration of the attack surface.
- **Attack** — confirm each vulnerability through controlled exploitation on the vuln build.
- **Verification & Reporting** — repeat each exploit against the secure build, document evidence, impact and the exact control that defeats it.

2. Test Environment Setup — Local DNS Mapping for Burp Suite Interception

Before any request tampering could be performed, the lab environment had to be prepared so that an intercepting proxy (Burp Suite) could reliably capture and modify traffic. By default, both Firefox and Burp Suite **bypass the proxy for `localhost` and `127.0.0.1`** traffic (the "do not proxy local addresses" behaviour), which means requests sent straight to `http://127.0.0.1:3000` never appear in Burp's HTTP history and cannot be intercepted.

The standard workaround is to give the loopback instance a **real-looking domain name** by adding a static entry to the local DNS resolver — the operating system's hosts file (`/etc/hosts`). Mapping the fictitious domain `meridian.tn` to `127.0.0.1` makes the browser treat it as a normal remote host (so the proxy is used), while the traffic still terminates on the local lab instance.

```
# edit the local DNS / hosts file
sudo nano /etc/hosts

# add this line (maps the lab domain to the loopback instance)
127.0.0.1    meridian.tn
```

```
ahmed@Dell-G15-5530 - [~/tekup/art_de_protection]
$ cat /etc/hosts
127.0.0.1 localhost
127.0.1.1 Dell-G15-5530

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
127.0.0.1 meridian.tn
```

*Figure 2.1 — The local hosts file (/etc/hosts) with the `127.0.0.1 meridian.tn` entry. This is why every request capture in this report shows **Host: meridian.tn** rather than localhost.*

The application is therefore reached at `http://meridian.tn:3000` (vulnerable) and `http://meridian.tn:3001` (secure) — the same local instances as `127.0.0.1:3000/3001`. With this mapping in place and Firefox pointed at Burp's listener on `127.0.0.1:8080`, every request is visible in Proxy → HTTP history and can be sent to Repeater or edited on the fly in Intercept.

3. Finding Severity Ratings & Risk Factors

The following table defines the severity levels and corresponding CVSS v3.1 score ranges used throughout this report.

Severity	CVSS v3 Range	Definition
Critical	9.0 – 10.0	Exploitation is straightforward and usually results in full compromise. Patch immediately.
High	7.0 – 8.9	Exploitation is more difficult but can cause elevated privileges, data loss or integrity loss. Patch as soon as possible.
Moderate	4.0 – 6.9	Vulnerabilities exist but are harder to exploit or require extra steps. Patch after high-priority issues.
Low	0.1 – 3.9	Non-exploitable issues that would reduce attack surface. Patch during the next window.
Info	N/A	No vulnerability. Information noted during testing or a strong observed control.

Risk Factors

Risk is measured by two factors: **Likelihood** and **Impact**.

Likelihood measures the potential of a vulnerability being exploited, based on the difficulty of the attack, available tooling, attacker skill and the environment. **Impact** measures the effect on confidentiality, integrity and availability of the application and its data, plus reputational and operational harm.

4. Scope

The assessment was limited to the two locally hosted builds of the Meridian Campus Portal:

- **Vulnerable build:** http://meridian.tn:3000 (`APP_MODE` default)
- **Secure build:** http://meridian.tn:3001 (`APP_MODE=secure`)
- **Stack:** Python/Flask backend, SQLite database, a static JavaScript single-page front-end, the `cryptography` library (AES-256-GCM + Ed25519) and a Solidity `GradeLedger` smart contract on a local `anvil` Ethereum node.
- **Environment:** Kali/Ubuntu Linux local lab, Firefox + Burp Suite Community.

Six seeded accounts were used for testing, spanning all three roles (admin, professor, student):

Username	Password	Role	Notes
admin	admin123	admin	id 1
prof_smith	teach123	professor	teaches PROG, DB, ALGO (id 2)
prof_jones	teach456	professor	teaches MATH, PHYS, NET (id 3)
alice	password1	student	id 4, enrolled in everything
bob	hunter2	student	id 5
carol	qwerty	student	id 6

Scope Exclusions

Denial-of-Service attacks and attacks against any system other than the two local lab instances were out of scope. No real users or third-party systems were involved. The source code was available to the tester (white-box), so findings cite exact file and line locations.

5. Executive Summary

This report summarises eleven vulnerabilities confirmed in the vulnerable build of the Meridian Campus Portal, spanning most of the OWASP Top 10 (2021) categories, and documents exactly how the hardened build neutralises each one. Several findings allow complete compromise of the application and its data.

Testing Summary

The most serious findings cluster around **injection** and **broken access control**. A SQL injection in the login form grants immediate administrator access with no valid credentials, and a second, UNION-based injection on the certifications endpoint dumps every user's password hash. Because those hashes are stored as **unsalted MD5**, five of six crack instantly in an online lookup. Authentication itself is undermined by a **broken JWT implementation** that never verifies the token signature, letting an attacker self-promote to `admin` by editing one claim. Broken access control recurs as two **IDOR** bugs and two **mass-assignment** privilege escalations (at registration and via the profile form). An **unrestricted file upload** enables stored XSS against the approving administrator, and a **path-traversal** flaw on the document-download endpoint leaks arbitrary server files — including the certificate signing key and the entire SQLite database.

Two findings are specifically about the integrity and authenticity controls that this application was built to showcase. In the vuln build, certificates are stored **unsigned and never anchored**, so a forged certificate injected via SQLi displays as "valid"; and grades have **no integrity anchoring** at all, so a direct database edit silently changes a student's record with nothing to flag it. The secure build answers both with a combination of **Ed25519 digital signatures** and an **on-chain hash ledger** (the GradeLedger smart contract): a record is trusted only if its signature verifies *and* its fingerprint is found on the blockchain, and any post-hoc tampering is detected and reversible from the ledger.

Key Weaknesses (vulnerable build)

1. User input is concatenated into SQL, enabling authentication bypass and data exfiltration.
2. JWTs are decoded but never cryptographically verified; `alg:none` is accepted.
3. Object- and function-level access control is missing (IDOR, mass assignment, privilege escalation).
4. Passwords are stored as unsalted MD5; admin responses leak the hashes.
5. Certificates are neither signed nor anchored, so forgeries are indistinguishable from genuine records.
6. Grades have no integrity reference, so database tampering is silent and undetectable.
7. File uploads are unrestricted (enabling stored XSS) and downloads are vulnerable to path traversal.

Collectively, these demonstrate that an attacker requires little skill or tooling to fully compromise the application, exfiltrate all user data, forge academic records and obtain administrative control — and, conversely, that the secure build's layered cryptographic and blockchain controls defeat each of these attacks.

6. Vulnerability Summary & Report Card

3	8	0	0	0
Critical	High	Moderate	Low	Info

#	Finding	Severity	Secure-build control
001	SQL Injection — Authentication Bypass	Critical	Parameterized queries; verified password hash
002	SQL Injection — UNION Hash Exfiltration	Critical	Parameterized queries; object-level authz; secrets never returned
003	Broken JWT — Admin Token Forgery	Critical	HMAC-SHA256 signature verified; `alg` pinned; role re-checked in DB
004	IDOR — Private Chats & Documents	High	Server-side ownership checks on every object
005	Mass Assignment — Registration Escalation	High	Role fixed to `student` server-side; body ignored
006	Cryptographic Failure — Unsalted MD5	High	PBKDF2-HMAC-SHA256, per-user salt, 200,000 iterations
007	Cryptographic Failure — Forged Certificates	High	Ed25519 signature + on-chain fingerprint anchoring
008	Data Integrity — Silent Grade Tampering	High	On-chain SHA-256 anchor; tamper monitor; ledger restore
009	Mass Assignment — Profile Escalation	High	Field allow-list; only UI fields written
010	Unrestricted File Upload → Stored XSS	High	PDF magic-byte check, 5 MB cap, random server name
011	Path Traversal — Local File Disclosure	Critical	Client path ignored; only the record's own file served

Severity reflects exploitability and impact on the vulnerable build; every finding is fully remediated in the secure build, as verified in Section 8.

7. Security Architecture of the Hardened Build (Cryptography & Blockchain)

Three findings in this report (WAPT-006, 007 and 008) are defended not by a one-line input check but by a layered **cryptographic + blockchain integrity architecture**. Because that architecture is the centrepiece of the application, it is described here once and referenced from each finding.

7.1 The trust problem

A student management system makes three security promises about academic records: **confidentiality** (private messages and documents stay private), **authenticity** (a certificate was really issued by the university) and **integrity** (a grade in the database is the grade the professor actually recorded). A conventional application stores all of this in a single SQLite database and trusts that database completely. That trust is the flaw: anyone who reaches the database — through SQL injection (WAPT-002/007), path traversal (WAPT-011) or direct shell access — can rewrite a grade or forge a certificate, and the application has **no independent reference** to contradict the tampered row. The secure build breaks this single point of trust by signing records with a key the database never holds, and by anchoring an immutable fingerprint of each record on a blockchain the database cannot reach.

Confidentiality

AES-256-GCM (authenticated encryption). Private message bodies are encrypted at rest with a 256-bit key kept in `keys/aes.key` (file mode `0600`, outside the DB). Each message uses a fresh 12-byte random nonce; the GCM authentication tag also detects any tampering of the ciphertext.

Authenticity

Ed25519 digital signatures. Each certificate is signed by the issuer's private key (`keys/issuer_ed25519.pem`). Anyone can verify with the public key, but only the university can produce a valid signature — so a certificate inserted by an attacker fails verification.

Integrity

GradeLedger smart contract on an Ethereum node. The SHA-256 fingerprint of every legitimate grade and certificate is anchored on-chain by an issuer-only key. The DB and the chain are compared on every read; any divergence is tamper.

7.2 Cryptographic primitives (crypto_utils.py / auth.py)

Control	Algorithm	Purpose & why it is sound
Password storage	PBKDF2-HMAC-SHA256, 200,000 iterations, 16-byte per-user salt	A unique salt defeats rainbow tables and makes identical passwords hash differently; 200k iterations slow each guess ~200,000× versus raw MD5, making wordlist attacks impractical. Verification uses constant-time comparison (`hmac.compare_digest`).
Encryption at rest	AES-256-GCM	Authenticated encryption: provides both confidentiality *and* integrity of the ciphertext. A random 96-bit nonce per message; the 128-bit GCM tag rejects any modified ciphertext on decrypt.
Certificate signatures	Ed25519 (EdDSA over Curve25519)	A modern, fast, misuse-resistant elliptic-curve signature. The issuer's private key signs a **canonical** JSON payload of the certificate; verification needs only the public key, so the web/database tier can verify but never forge.

Token integrity	HMAC-SHA256 (JWT HS256)	The session JWT is signed with a server secret; the secure decoder pins `alg=HS256`, verifies the MAC in constant time and checks `exp` before trusting any claim.
On-chain fingerprint	SHA-256	The 32-byte hash of a canonical grade/certificate payload is what gets stored on-chain — never the plaintext grade or any personal data.

The **canonical payload** is the detail that makes signatures and anchors meaningful. A certificate is serialised with sorted keys and no whitespace before it is signed or hashed:

```
# crypto_utils.py — the exact bytes that get signed AND hashed
def cert_payload(student_id, title, grade, issued_date) -> bytes:
    return json.dumps(
        {"student_id": student_id, "title": title,
         "grade": grade, "issued_date": issued_date},
        sort_keys=True, separators=(",", ":"),
    ).encode()

def cert_fingerprint(...):   return sha256(cert_payload(...)).hexdigest()   # anchored
on-chain
def sign_cert(...):          return ed25519_private.sign(cert_payload(...)) # stored in
the DB
```

Because the signature and the on-chain fingerprint are both bound to this canonical payload, changing *any* field — the grade, the title, the student — breaks the signature **and** changes the fingerprint, so the record no longer matches the chain. The attacker cannot re-sign (no private key) and cannot re-anchor (the contract rejects non-issuer writers).

7.3 The blockchain integrity ledger (GradeLedger.sol)

The ledger is a Solidity smart contract deployed to a local Ethereum development node (`anvil`, chain id `31337`). It is deliberately minimal: it stores only **hashes**, never names or grades, and it accepts writes from exactly one account — the issuer. This is the independent reference the SQLite database is checked against.

```
// contracts/src/GradeLedger.sol  (abridged)
contract GradeLedger {
    address public immutable issuer;                // fixed forever at deployment
    mapping(uint => mapping(uint => bytes32)) private grades;      // course => student
=> hash
    mapping(bytes32 => bool) private certificates;                 // fingerprint =>
anchored

    modifier onlyIssuer() {                          // THE key control
        if (msg.sender != issuer) revert NotIssuer();
        _;
    }
    function anchorGrade(uint c, uint s, bytes32 h) external onlyIssuer { grades[c][s] =
h; ... }
    function anchorCertificate(bytes32 fp)          external onlyIssuer
{ certificates[fp] = true; ... }
    function getGrade(uint c, uint s) external view returns (bytes32) { return grades[c]
[s]; }
```

```
    function isCertificateAnchored(bytes32 fp) external view returns (bool) { ... }
}
```

Why the blockchain, and not just a second database table? Three properties the SQLite file cannot provide on its own:

- **Write authorisation by cryptographic key, not by application code.** The `onlyIssuer` modifier is enforced by the Ethereum network itself. An attacker who fully owns the web server and the SQLite file still cannot anchor a forged record, because they do not hold the issuer's Ethereum private key (`keys/eth_issuer.key` , mode `0600`). Contrast this with a second DB table, which that same attacker would simply rewrite.
- **Append-only, tamper-evident history.** Every anchor is a transaction in an immutable, hash-linked block; the admin "Anchored events" explorer lists each one with its block number and transaction hash. There is no UPDATE or DELETE.
- **Separation of duties.** The anchoring key lives in the application layer only long enough to sign a transaction; the record of truth lives on a separate system with a different trust model. Compromising one does not compromise the other.

The application talks to the contract through `eth_ledger.py` , which refuses any non-loopback RPC endpoint and any chain id other than 31337, funds a freshly generated issuer key from the dev faucet, and exposes four operations: `anchor_grade` , `get_grade` , `anchor_cert` , `is_cert_anchored` . If the node is unreachable it raises `LedgerUnavailable` , which the API turns into an HTTP 503 — the secure build **fails closed**: it will not issue a certificate it could not anchor.

7.4 Recovering the true value from a hash

The contract stores only a SHA-256 hash, so there is no "read the grade back from the chain" call. Yet the admin tamper monitor shows the *correct* grade beside a tampered one (see WAPT-008). This works because grades live on a tiny, known domain — 0.00 to 20.00 in 0.25 steps, 81 possible values. The secure build simply hashes each candidate and compares it to the anchored hash to recover the real value:

```
# eth_ledger.py — recover the anchored grade by exhaustive match over a tiny domain
for step in range(0, 81):                       # 0.00, 0.25, ..., 20.00
    candidate = f"{step * 0.25:.2f}"
    if grade_hash(course_id, student_id, candidate) == target_chain_hash:
        return candidate
```

> **NOTE**
>
> This brute-force recovery is sound **only because** the input space is small and the hash is unsalted. It is a deliberate design choice for a non-confidential value (a grade on a fixed scale); a field that needed confidentiality would be anchored with a salted commitment instead, which this report notes as the correct pattern for secret data.

8. Technical Findings

Each finding below includes a description, OWASP Top 10 (2021) mapping, a risk rating (likelihood and impact), the affected component with source references, tools used, references, a reproducible proof-of-concept with evidence, and — the focus of this assessment — the exact control in the secure build that defeats the attack. All captures were taken on the tester's own local instances.

Finding WAPT-001: SQL Injection — Authentication Bypass

Critical

Description. The login handler builds its SQL query by concatenating the client-supplied username directly into the statement, with no parameterization. A crafted username comments out the password check, authenticating the attacker as the first matching row — the administrator — without any password.

OWASP Top 10 (2021)	A03:2021 — Injection
Risk — Likelihood	High — a single, well-known payload typed into the login form; no tooling required.
Risk — Impact	Very High — complete administrator takeover, exposing all user data and privileged functions.
Affected Component	`POST /api/login` (username field) — login SQL built by string concatenation (vuln build).
Tools Used	Web browser, Burp Suite
References	OWASP A03 Injection; WSTG-INPV-05; OWASP SQL Injection Prevention Cheat Sheet

Proof of concept

On the vulnerable login page, submit the following and any password:

```
Username:  admin'--  (note the trailing space after --)
Password:  anything
```

The trailing `--` turns the rest of the query (the password comparison) into a comment, so the server authenticates the `admin` row directly. The dashboard loads as **admin** with no valid credentials.

> **SECURE BUILD (MERIDIAN.TN:3001) — THE FIX**
>
> The secure build uses a parameterized query (`... WHERE username = ?`) so the payload is treated as a literal username string that does not exist, and the supplied password is verified against the stored PBKDF2 hash. The same input returns **“invalid username or password”**.

Finding WAPT-002: SQL Injection — UNION-based Password-Hash Exfiltration Critical

Description. The certifications endpoint concatenates the `student_id` query parameter into a numeric SQL context with no validation and no authorization check. An attacker logged in as an ordinary student can inject a `UNION SELECT` to read arbitrary tables — including every user's password hash — and render them straight into the Certifications table in the browser.

OWASP Top 10 (2021)	A03:2021 — Injection (with A01 Broken Access Control)
Risk — Likelihood	High — exploitable from the address bar by any authenticated student.
Risk — Impact	Very High — full database read: all users' password hashes, enabling mass account compromise after cracking.
Affected Component	`GET /api/certifications?student_id=` — `app.py:280-288` (vuln build).
Tools Used	Web browser, Burp Suite, hashcat / CrackStation
References	OWASP A03 Injection; WSTG-INPV-05

Proof of concept

Logged in as `alice`, browse to the Certifications page, then drive the injection from the address bar. `?student_id=0 OR 1=1` returns every student's certificates; the UNION payload exfiltrates the user table:

```
/#certifications?student_id=0 union select 1,2,password_hash,4,5,6,7,8 from users--
```

The six MD5 digests render directly in the `grade` / `title` columns of the Certifications table. These hashes feed directly into WAPT-006 (cracking).

> **SECURE BUILD (MERIDIAN.TN:3001) — THE FIX**
>
> Two independent controls: the query is **parameterized** (`WHERE c.student_id = ?`), so `OR 1=1` / `UNION` are inert text; and an **object-level authorization check** ensures a student may read only their own `student_id` — any other id returns **403**. Password hashes are never selected into this response in the first place.

Finding WAPT-003: Broken JWT — Admin Token Forgery

Critical

Description. After login the server issues a JSON Web Token (JWT) carrying the user's `sub`, `username` and `role`. The vulnerable build **decodes the token but never verifies its signature** (`jwt_decode_vuln`, `auth.py`), and even accepts `alg:none`. Privileged endpoints trust the `role` claim straight from the token. An attacker can therefore flip `"role":"student"` to `"admin"` in their own token — with no re-signing — and the server believes it.

OWASP Top 10 (2021)	A02:2021 — Cryptographic Failures (signature not verified) / A07 Identification & Authentication Failures
Risk — Likelihood	High — a logged-in user edits one claim in Burp's JWT editor; the signature is left untouched or deleted.
Risk — Impact	Very High — horizontal and vertical privilege escalation to admin; exposes the full user list with every password hash.
Affected Component	JWT verification in `auth.py:jwt_decode_vuln`; admin endpoint trusting the role claim (`app.py`).
Tools Used	Firefox, Burp Suite (Proxy / Repeater / JWT Editor)
References	OWASP A02 Cryptographic Failures; A07; WSTG-SESS-10; RFC 7519

Proof of concept

1. With Firefox proxied through Burp, log in as `alice` / `password1`. In Proxy → HTTP history find an authenticated request (`GET /api/me` carries the `Authorization: Bearer` header) and send it to **Repeater**.
2. Aim it at the admin endpoint — change the first line to `GET /api/admin/users HTTP/1.1` and Send. You get `403 {"error":"admins only"}`: the token is accepted, you are just not admin yet.
3. In the **JSON Web Token** tab, change `"role":"student"` to `"role":"admin"`. **Leave the signature alone.** Send → **200 OK** and the full user list, including every `password_hash`.

```
{ "users": [ { "id":1, "username":"admin", "role":"admin",
               "password_hash":"0192023a7bbd73250516f069df18b500", ... }, ... ] }
```

The signature is irrelevant: deleting the entire third segment (leaving a trailing dot) still returns 200. Setting `"sub":1` promotes the whole UI to admin, since `/api/me` loads the user by `sub`.

> **SECURE BUILD (MERIDIAN.TN:3001) — THE FIX**
>
> The secure decoder (`jwt_decode_secure`) **pins `alg=HS256`**, recomputes the HMAC-SHA256 over the header and payload and compares it in constant time, and rejects expired tokens. The edited token fails the MAC check and returns **401**. As defence in depth, the admin endpoint re-loads the caller's role **from the database** rather than trusting the token claim, and never returns password hashes.

Finding WAPT-004: IDOR — Private Chats & Student Documents High

Description. Two insecure-direct-object-reference bugs share one pattern: the object is selected by an id in the request, with no check that the caller may access it. A student can read any private conversation and any other student's official document by changing an id.

OWASP Top 10 (2021)	A01:2021 — Broken Access Control (IDOR)
Risk — Likelihood	High — ids are visible in the user's own URLs; the attack is a single value change.
Risk — Impact	High — disclosure of confidential conversations (including a leaked master password) and other students' personal data.
Affected Component	`GET /api/documents/<id>` (`app.py:875-893`); chat retrieval by `chat_users[]` .
Tools Used	Web browser, Burp Suite
References	OWASP A01; WSTG-ATHZ-04 (IDOR)

Proof of concept — 4a: read anyone's private chat

As `alice` , open Chat and a conversation; the URL shows `#chat?chat_users[0]=4&chat_users[1]=5` . Change it to read the private admin ↔ professor conversation:

```
#chat?chat_users[0]=1&chat_users[1]=2
```

The conversation is disclosed, including the line “the grading portal master password is `Adm1n_Gu@rd14n_2026` ”.

Proof of concept — 4b: read another student's document

```
// address bar  #documents?doc=1
// or console  call('GET','/api/documents/1')
```

Alice sees another student's document — name, e-mail and reference number.

> **SECURE BUILD (MERIDIAN.TN:3001) — THE FIX**
>
> Both endpoints add a server-side ownership check: a student may read a document or a conversation only if they are a participant/owner; staff and admins are allowed by role. Any other id returns **403 forbidden**.

Finding WAPT-005: Mass Assignment — Privilege Escalation at Registration High

Description. The sign-up form sends only `full_name`, `username`, `email` and `password`. The vulnerable build, however, reads a `role` from the request body if one is present and writes it straight into the new user row. A new account is a `student` only because the form omits `role` — an attacker who adds the field chooses their own role.

OWASP Top 10 (2021)	A01:2021 — Broken Access Control (Mass Assignment)
Risk — Likelihood	High — a single intercepted registration request with one extra JSON field.
Risk — Impact	High — immediate creation of an attacker-controlled administrator account.
Affected Component	`POST /api/register` — `role = b.get("role","student")` (`app.py:140-148` , vuln build).
Tools Used	Firefox, Burp Suite (Intercept)
References	OWASP A01; WSTG-BUSLOGIC; Mass Assignment Cheat Sheet

Proof of concept

With Intercept on, submit the registration form and edit the captured body to add a role:

```
{ "full_name":"ahmed", "username":"ahmed", "email":"ahmed@campus.edu",
  "password":"ahmed123", [redacted] }
```

Forward the request, then log in as `ahmed` / `ahmed123`: the dashboard greets “Administrator dashboard” and the admin menus appear.

> **SECURE BUILD (MERIDIAN.TN:3001) — THE FIX**
>
> The secure build fixes `role = "student"` on the server and ignores any role in the body (`app.py:143`). The same tampered request produces a plain student account.

Finding WAPT-006: Cryptographic Failure — Unsalted MD5 Password Storage High

Description. The vulnerable build stores passwords as **unsalted MD5** digests. MD5 is fast and broken for password hashing, and the absence of a salt means identical passwords share a hash and precomputed rainbow tables / online lookups apply directly. Combined with the hash leaks in WAPT-002/003, this converts a database read into full credential compromise.

OWASP Top 10 (2021)	A02:2021 — Cryptographic Failures
Risk — Likelihood	High — once hashes leak, cracking is instant with free online tables; no GPU needed.
Risk — Impact	High — plaintext recovery of most user passwords, enabling account takeover across roles.
Affected Component	Password hashing in `auth.py:hash_password` / `verify_password`.
Tools Used	hashcat, CrackStation, sqlite3
References	OWASP A02; WSTG-ATHN; OWASP Password Storage Cheat Sheet; NIST SP 800-63B

Proof of concept

Take the six MD5 digests leaked via WAPT-002 and crack them — e.g. paste into CrackStation or run hashcat:

```
hashcat -m 0 -a 0 hashes.txt rockyou.txt
```

Five of six crack instantly (`admin123` , `hunter2` , `password1` , `teach123` , `qwerty`) — the sixth simply is not in that wordlist. Cracking is instant precisely because MD5 is cheap and unsalted.

SECURE BUILD (MERIDIAN.TN:3001) — THE FIX

The secure build hashes with **PBKDF2-HMAC-SHA256**, a **unique 16-byte salt per user** and **200,000 iterations**, verified in constant time. The salt kills rainbow tables and makes identical passwords hash differently; the iteration count makes each guess ~200,000× slower, so a wordlist attack is impractical. Confirm with:

```
sqlite3 sms_secure.db "SELECT username,hash_scheme,salt FROM users"
# scheme = pbkdf2, a distinct salt per row
```

Finding WAPT-007: Cryptographic Failure — Unsigned & Un-anchored Certificates (Forgery) High

Description. Academic certifications are the application's highest-value authenticity asset. The vulnerable build stores a certificate **without any digital signature** and **never anchors it**, and on read it is reported "verified" with no check at all (`c["verified"] = True`, `app.py:299-302`). Consequently a certificate an attacker injects directly — e.g. via the stacked-query SQLi below — displays as a genuine, valid credential. This is the exact threat the secure build's Ed25519 + blockchain layer (Section 7) is designed to defeat.

OWASP Top 10 (2021)	A02:2021 — Cryptographic Failures / A08:2021 — Software & Data Integrity Failures
Risk — Likelihood	Medium–High — requires an injection or DB write, but the forged record is then indistinguishable from a real one.
Risk — Impact	High — forged academic credentials (e.g. a fake "PhD") presented as university-verified; reputational and legal harm.
Affected Component	Certificate issue/verify path — `app.py:299-315` (read), `426-450` (issue); `crypto_utils.verify_cert`; `eth_ledger.is_cert_anchored`.
Tools Used	Burp Suite, sqlite3, browser
References	OWASP A02 / A08; WSTG-CRYP; NIST FIPS 186-5 (EdDSA)

Proof of concept — forge a "valid" certificate

As `alice`, inject an unsigned certificate through the Course-search box with a stacked-query SQLi (no DB access needed on the vuln build):

```
%'; INSERT INTO certifications (student_id,title,grade,issued_date,signature,issued_by)
       VALUES (4,'PhD in Computer Science','A+','2026-01-01',NULL,1); --
```

The search shows no matches, but the Certifications page now lists a "PhD in Computer Science" marked **valid** — a forged credential accepted by the vuln build because it never checks a signature or the ledger.

How the secure build defeats it (Ed25519 signature + on-chain anchor)

In the secure build, issuing a certificate signs its canonical payload with the issuer's Ed25519 private key and anchors the SHA-256 fingerprint on the GradeLedger contract; the row is committed only if the on-chain anchor succeeded (otherwise HTTP 503 — fail closed). On **read**, two independent checks must both pass:

```
# app.py (secure) — a certificate is trusted only if BOTH hold
c["verified"] = verify_cert(student_id, title, grade, issued_date, signature)  # Ed25519
fp            = cert_fingerprint(student_id, title, grade, issued_date)
c["on_chain"] = is_cert_anchored(fp)                                           #
blockchain
```

A certificate inserted straight into the secure database has `signature = NULL` (the attacker cannot produce a valid Ed25519 signature without the private key) **and** a fingerprint that was never anchored (the attacker cannot write to the contract without the issuer's Ethereum key). It therefore shows as **✗ invalid** and **not on chain**. Reproduce the DB-level forgery against the secure build:

```
sqlite3 sms_secure.db "INSERT INTO certifications
(student_id,title,grade,issued_date,signature,issued_by) VALUES (4,'PhD in Computer
Science','17','2026-01-01',NULL,1)"
```

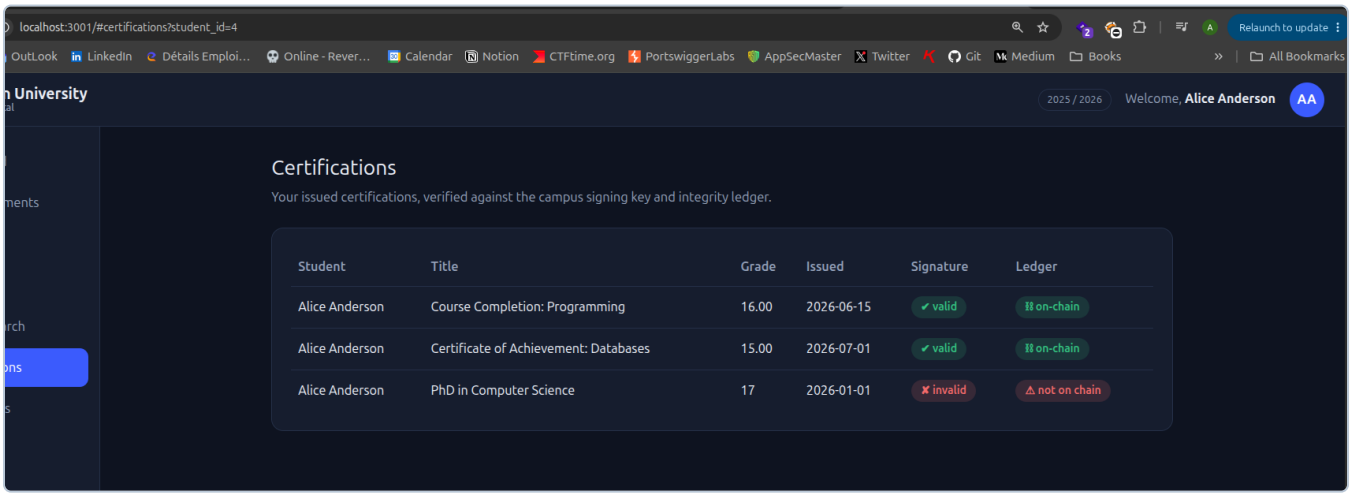


*Figure 7.1 — The same forged certificate on the **secure** build: the signature fails Ed25519 verification and the fingerprint is absent from the ledger, so it is flagged **invalid** and **not on chain**. The vuln build showed the identical record as "valid".*

SECURE BUILD (MERIDIAN.TN:3001) — THE FIX

Authenticity is anchored in two keys the attacker never holds: the **Ed25519 signing key** (so forgeries fail signature verification) and the **issuer's Ethereum key** (so forgeries are absent from the immutable ledger). Both must succeed for a certificate to read as genuine.

Finding WAPT-008: Data Integrity Failure — Silent Grade Tampering High

Description. The vulnerable build keeps grades in SQLite with **no integrity reference**. Anyone who reaches the database — via SQLi, path traversal (WAPT-011) or shell access — can rewrite a grade and nothing flags it, because the application has no independent record of the correct value. The secure build anchors the SHA-256 hash of every legitimate grade on the GradeLedger contract and compares the database against the chain on every read, so any out-of-band edit is detected and reversible.

OWASP Top 10 (2021)	A08:2021 — Software & Data Integrity Failures
Risk — Likelihood	Medium — requires database access, but that access is reachable through other findings in this report.
Risk — Impact	High — undetected alteration of official academic records (grade inflation / sabotage) with no audit trail.
Affected Component	Grade write/verify — `app.py:491-507` (anchor on write), `645-776` (tamper monitor); `GradeLedger.anchorGrade/getGrade`.
Tools Used	sqlite3, Ethereum node (anvil), browser
References	OWASP A08; NIST SP 800-57 (key management)

Proof of concept — tamper with a grade directly in the database

Simulate an attacker who has reached the database and inflates Alice's Mathematics grade to the maximum:

```
sqlite3 sms_vuln.db   "UPDATE grades SET grade='20.00' WHERE student_id=4 AND
course_id=1;"
sqlite3 sms_secure.db "UPDATE grades SET grade='20.00' WHERE student_id=4 AND
course_id=1;"
```

Vulnerable build: log in as `alice` — the dashboard simply shows 20/20. Nothing anywhere indicates the record was altered.

How the secure build detects and reverses it (on-chain anchor)

Every legitimate grade change anchors its hash on the contract (`anchor_grade(course_id, student_id, grade_hash)`), committed only if the transaction succeeded. On read, the build recomputes the hash of the stored grade and compares it with the anchored hash; a mismatch is tamper. The **true** value is recovered from the on-chain hash by matching candidates on the 0–20 scale (Section 7.4).

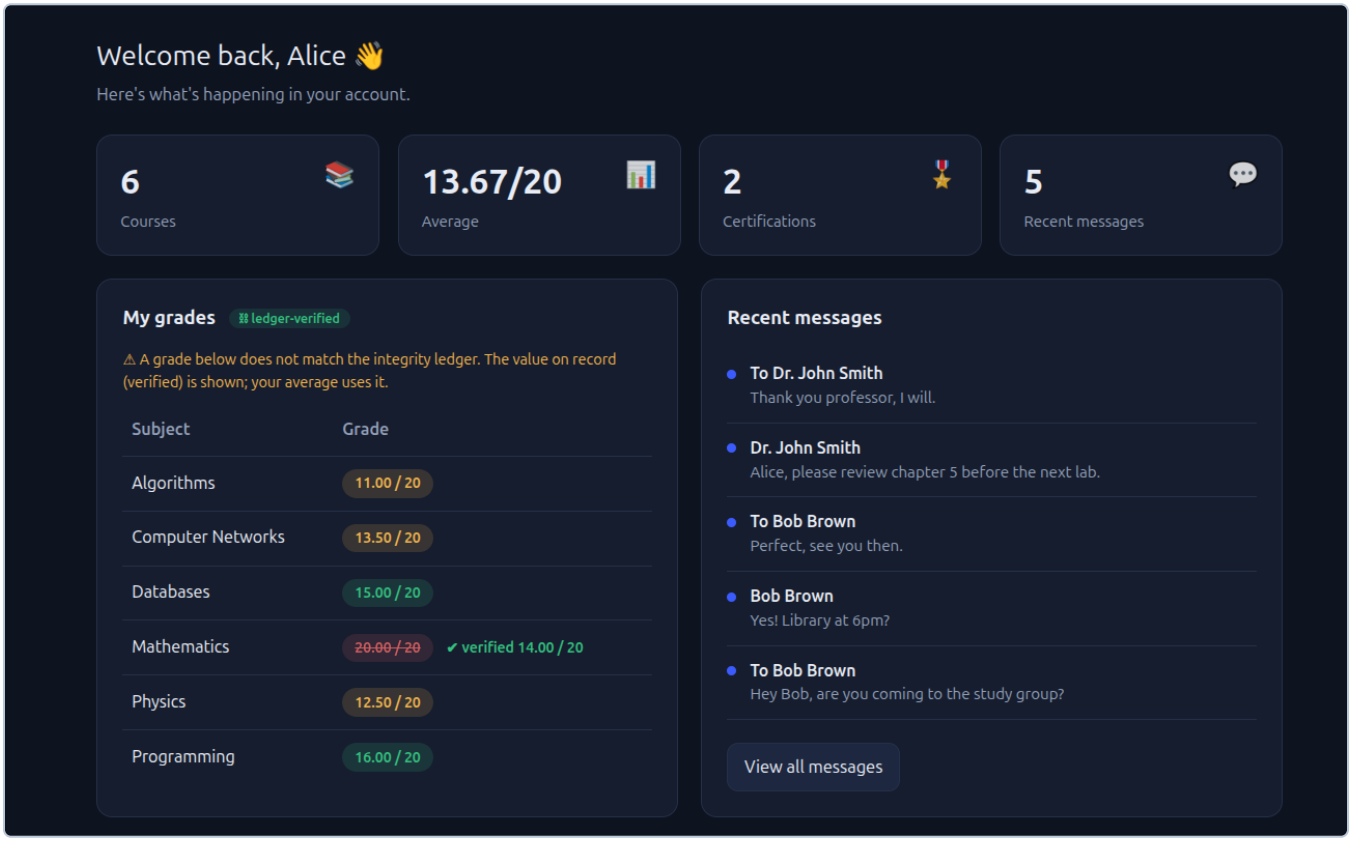


*Figure 8.1 — The student dashboard on the **secure** build after the same DB edit: the tampered value (20.00) is struck through in red and the **verified** grade recovered from the ledger is shown beside it; the computed average is unaffected.*

The administrator's **Integrity & ledger** console raises a live alert and offers a one-click restore that rewrites the database back to the anchored value:

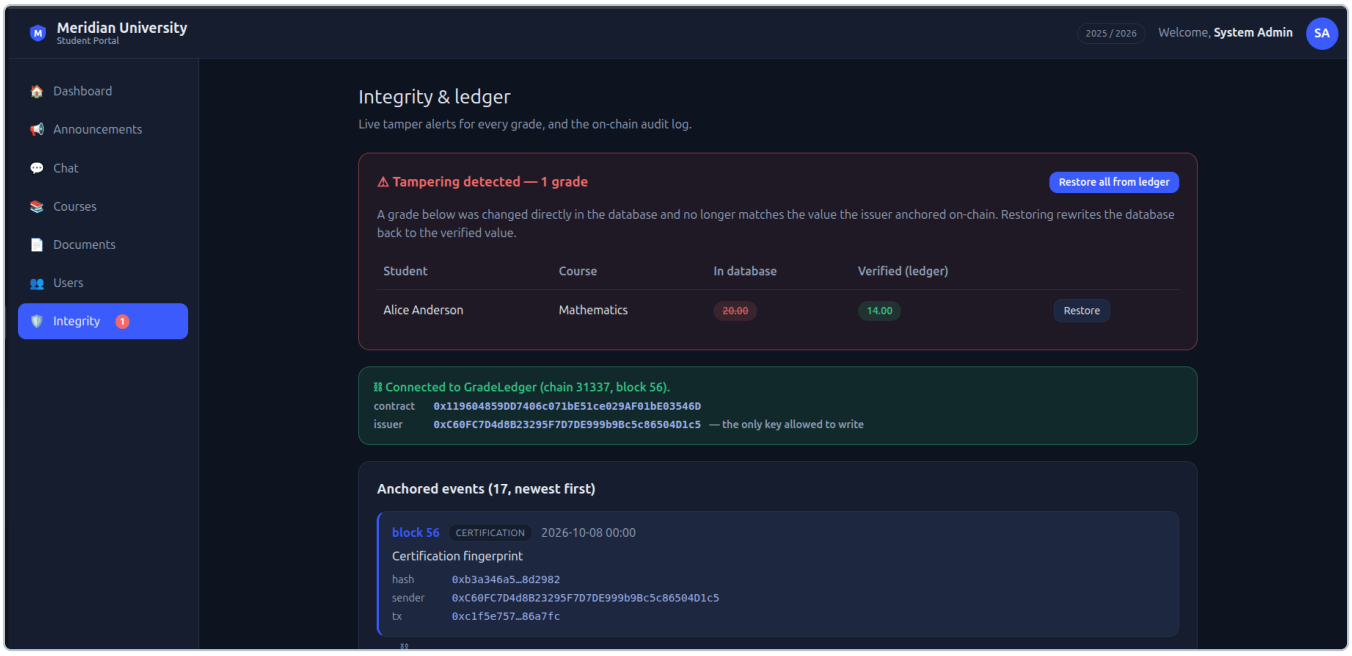


*Figure 8.2 — Admin Integrity console: "Tampering detected — 1 grade" for Alice Anderson / Mathematics (In database 20.00 vs Verified-ledger 14.00), with **Restore all from ledger**. Below: "Connected to GradeLedger (chain 31337)", the contract address `0x1196…546D`, the issuer key ("the only key allowed to write") and the append-only anchored-event log with per-event transaction hashes.*

SECURE BUILD (MERIDIAN.TN:3001) — THE FIX

Integrity is enforced by comparing the database against an **independent, append-only on-chain anchor** written only by the issuer key. A direct DB edit changes the hash, so DB ≠ chain and the tamper is flagged instantly; **Restore all from ledger** rewrites the DB back to the verified value. The attacker cannot hide the edit because they cannot re-anchor the forged value without the issuer's Ethereum key.

Finding WAPT-009: Mass Assignment — Privilege Escalation from Profile Update High

Description. The Profile form edits only name, e-mail, phone and address, but the vulnerable build writes **any** field present in the request body onto the user's own row — the textbook mass-assignment flaw. A student can add `role` and promote themselves to administrator.

OWASP Top 10 (2021)	A01:2021 — Broken Access Control (Mass Assignment)
Risk — Likelihood	High — one intercepted “Save Changes” request with an added field.
Risk — Impact	High — self-service privilege escalation from student to admin.
Affected Component	Profile update handler (vuln build writes arbitrary body keys to the user row).
Tools Used	Firefox, Burp Suite (Intercept)
References	OWASP A01; WSTG-BUSLOGIC; Mass Assignment Cheat Sheet

Proof of concept

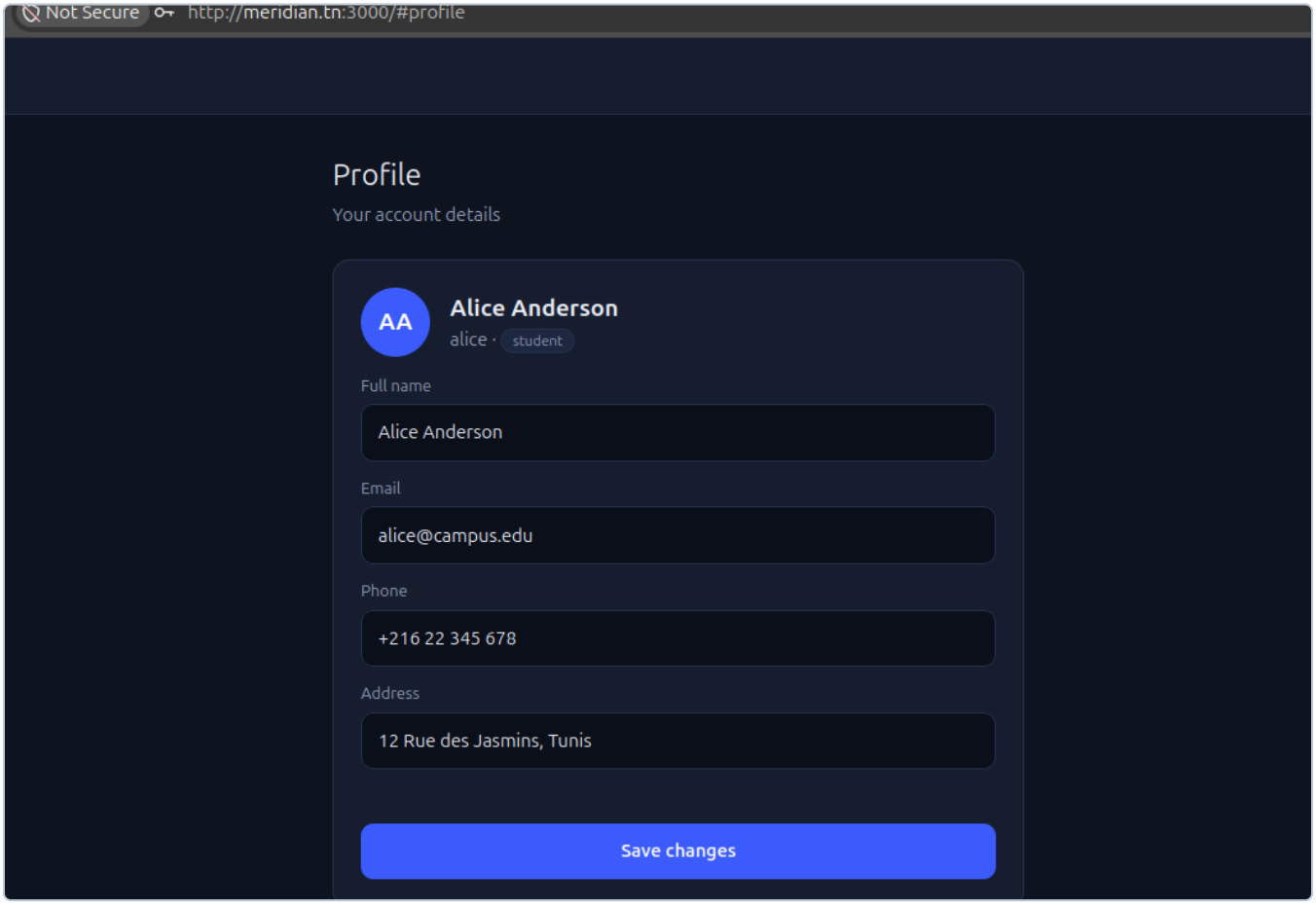


Figure 9.1 — The Profile “Save Changes” request captured in Burp; the body carries only the visible profile fields.

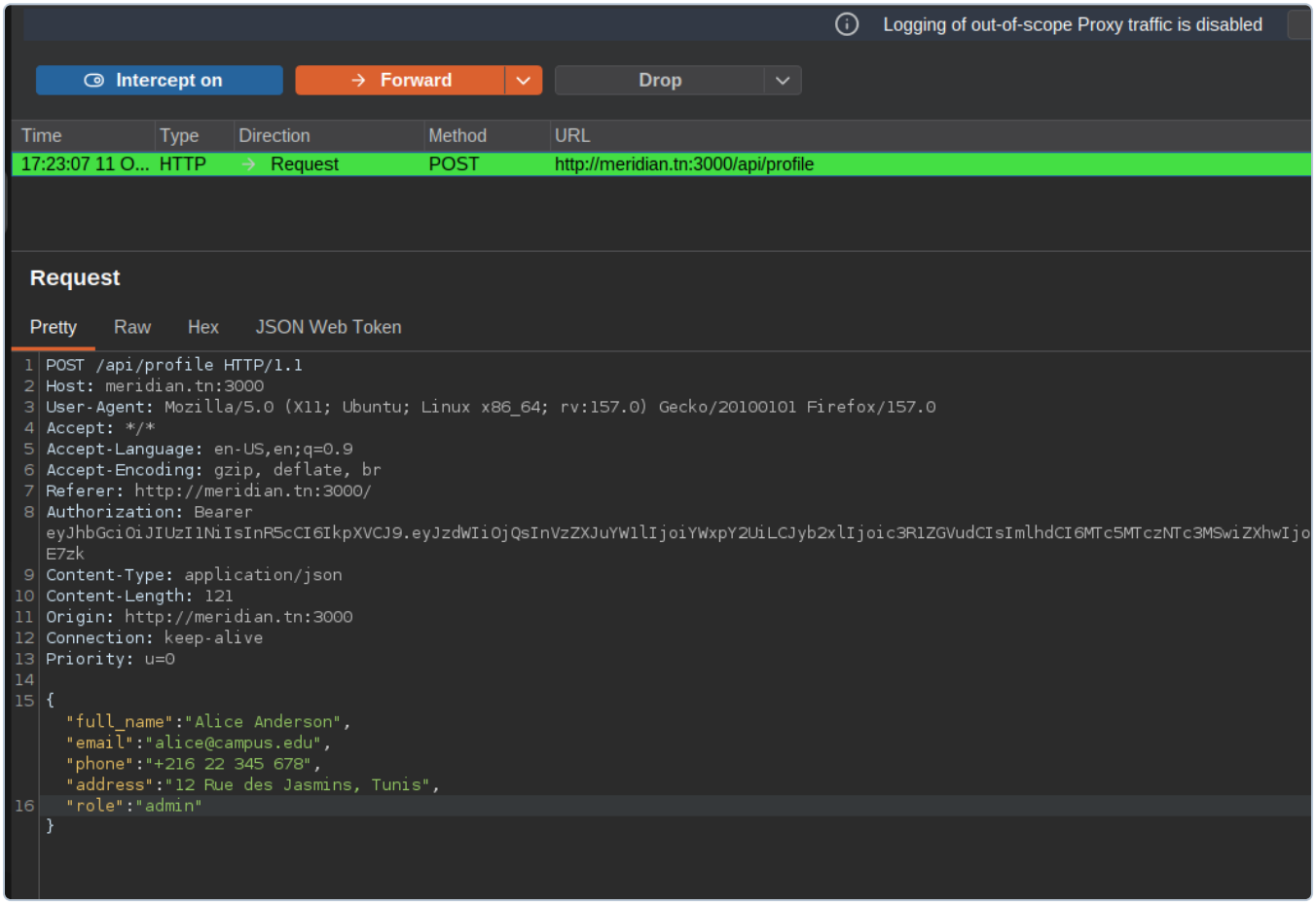


Figure 9.2 — The intercepted request edited to add `"role":"admin"` before forwarding. On the next login the account has full administrator access.

SECURE BUILD (MERIDIAN.TN:3001) — THE FIX

The secure build writes only an explicit **allow-list** of the fields the UI actually exposes (name, e-mail, phone, address). `role` and any other unexpected key are ignored, so the user stays a student.

Finding WAPT-010: Unrestricted File Upload → Stored XSS

High

Description. The document-request upload accepts **any file, of any type and size, stored verbatim under the client-supplied filename**. A student can upload an SVG/HTML file containing JavaScript; when an administrator opens it during approval, the script executes in the admin's authenticated session (stored XSS), enabling theft of the admin JWT and full account takeover.

OWASP Top 10 (2021)	A05:2021 — Security Misconfiguration / A03 Injection (Stored XSS) / A08 Integrity
Risk — Likelihood	Medium–High — a student uploads a crafted file; execution waits for an admin to view it.
Risk — Impact	High — stored XSS in an admin context → session/JWT theft → administrative takeover.
Affected Component	Upload handler `_save_upload` (`app.py:816-839` , vuln branch).
Tools Used	Web browser, Burp Suite
References	OWASP A05 / A03; WSTG-BUSLOGIC-09 (upload); File Upload Cheat Sheet

Proof of concept

As `alice`, create a non-PDF “document” carrying an XSS payload and submit it as a document request:

```
printf '                                        ' > evil.svg
```

The vuln build accepts and stores it verbatim, under the attacker's chosen filename. When the administrator opens the attachment to approve the request, the payload runs in the admin origin — in a real attack, exfiltrating the admin's JWT rather than calling `alert()`.

> **SECURE BUILD (MERIDIAN.TN:3001) — THE FIX**
>
> The secure build **ignores the client filename**, requires a genuine PDF by checking the `%PDF-` magic bytes, caps the size at 5 MB, and stores the file under a random server-generated name (`secrets.token_hex(16)+".pdf"`) inside the upload directory. Non-PDF content is rejected with “only PDF files are accepted”, so no active content can be stored or served.

Finding WAPT-011: Path Traversal — Local File Disclosure

Critical

Description. The document-download endpoint serves whatever filename the client passes in `?name=`, joined onto the uploads directory with **no sanitisation**. By supplying `../` sequences, an attacker escapes the uploads folder and downloads arbitrary server files — including the certificate signing key and the entire SQLite database, which directly undermines every other control in the application.

OWASP Top 10 (2021)	A01:2021 — Broken Access Control (Path Traversal) / A05 Misconfiguration
Risk — Likelihood	High — an authenticated user appends a crafted query parameter to a normal download URL.
Risk — Impact	Critical — disclosure of the Ed25519 signing key and the full database (all hashes, messages, grades); enables certificate forgery and total compromise.
Affected Component	`GET /api/documents/<id>/download?name=` (`app.py:896-918` , vuln branch).
Tools Used	Web browser, Burp Suite
References	OWASP A01 / A05; WSTG-ATHZ-01 (Directory Traversal); Path Traversal Cheat Sheet

Proof of concept

Step 1. When a student views an uploaded document, a request is sent that returns the PDF in the response:

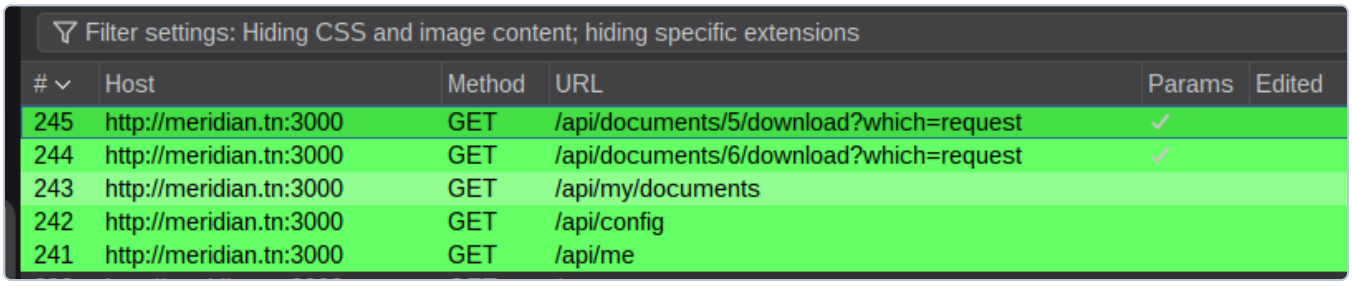


Figure 11.1 — The normal document-download request that returns the student's own uploaded PDF.

Step 2. Fuzzing the query parameters shows that `?name` changes the response — indicating the value is used to build the served file path:

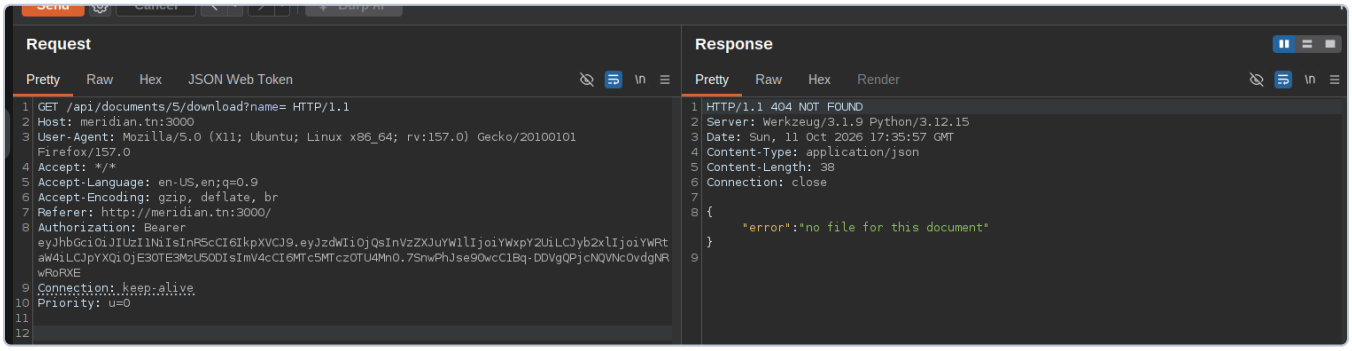


Figure 11.2 — Probing parameters: only `?name` alters the response, revealing it is processed server-side to locate the file.

Step 3. Supplying a traversal path discloses arbitrary server files. Downloading the whole database:

```
GET /api/documents/5/download?name=../../sms_vuln.db HTTP/1.1
Host: meridian.tn:3000
```

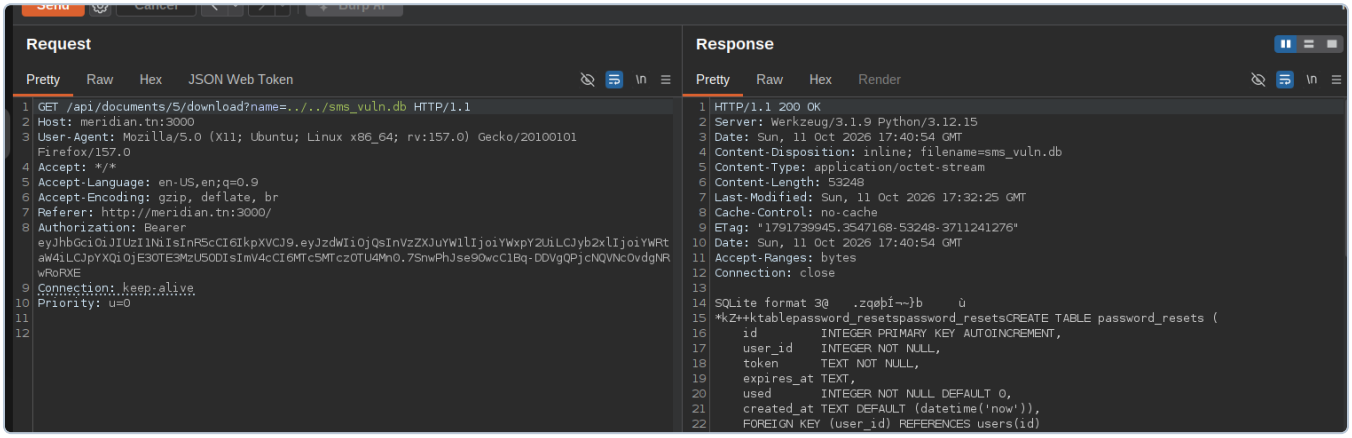


*Figure 11.3 — Path traversal confirmed: `?name=../../sms_vuln.db` returns **200 OK** with the raw SQLite database (note `SQLite format 3` in the body and `filename=sms_vuln.db`). The same technique with `?name=../../keys/issuer_ed25519.pem` leaks the certificate signing key.*

SECURE BUILD (MERIDIAN.TN:3001) — THE FIX

The secure build **ignores the `name` parameter entirely** and serves only the file recorded for that specific document, with a fixed `application/pdf` content type and an ownership check (students may download only their own). The client never influences the served path, so traversal is impossible.

Appendix A — Methodology & Tools

Testing followed the OWASP Web Security Testing Guide (WSTG) and mapped findings to the OWASP Top 10 (2021). Each vulnerability was first confirmed on the vulnerable build, then re-tested against the secure build to verify the control.

Tool	Use in this assessment
Firefox + Burp Suite (Proxy, Repeater, Intercept, JWT Editor)	Request interception and tampering: JWT forgery, mass assignment, path traversal.
Local DNS / hosts file (`meridian.tn → 127.0.0.1`)	Route loopback traffic through Burp (Section 2).
sqlite3	Simulate database-level tampering and inspect password schemes/salts.
hashcat / CrackStation	Crack the leaked unsalted MD5 password hashes.
anvil (Foundry) + web3.py	Local Ethereum node hosting the GradeLedger contract; anchor/verify operations.
Browser DevTools console	Drive IDOR and API calls directly (`call('GET', ...)`).

Appendix B — Cryptographic & Blockchain Reference

Mechanism	Where	Parameters
Password KDF	`auth.py`	PBKDF2-HMAC-SHA256, 200,000 iterations, 16-byte per-user salt, constant-time verify
JWT	`auth.py`	HS256; secure decoder pins alg, verifies HMAC in constant time, checks `exp`
Encryption at rest	`crypto_utils.py`	AES-256-GCM, random 96-bit nonce, 128-bit tag; key in `keys/aes.key` (0600)
Certificate signature	`crypto_utils.py`	Ed25519 over canonical JSON; private key in `keys/issuer_ed25519.pem` (0600)
On-chain anchor	`eth_ledger.py` / `GradeLedger.sol`	SHA-256 fingerprint; `onlyIssuer` writes; chain id 31337; fail-closed (503 if node down)

End of report — BUSINESS CONFIDENTIAL — TEK-UP / Web Security Module 2025-2026.